

```

#include "artery/application/TrajektoriAppVerIND.h" // <- Ganti
#include "artery/application/VehicleDataProvider.h"
#include "artery/application/CaService.h"
#include "artery/application/Middleware.h"
#include <vanetza/asn1/its/CAM.h>
#include <vanetza/asn1/cam.hpp>
#include <omnetpp.h>
#include <cmath>
#include <iomanip>
#include <vector>

namespace artery {
using namespace omnetpp;

Define_Module(TrajektoriAppVerIND); // <- Ganti

// Fungsi ini sama persis
std::string TrajektoriAppVerIND::getNodeType()
{
    std::string type = getParentModule()->getNedTypeName();
    if (type.find("Vehicle") != std::string::npos) return "Vehicle";
    if (type.find("Person") != std::string::npos) return "Person";
    return "Unknown";
}

void TrajektoriAppVerIND::initialize()
{
    // --- Logika Inisialisasi Log Mentah (Sama seperti v1) ---
    ItsG5BaseService::initialize();
    mLogInterval = par("logInterval");
    std::string nodeType = getNodeType();
    std::string logFilename;
    if (nodeType == "Vehicle") {
        logFilename = "results/CAM_data_from_car_v2.csv"; // (v2)
    } else {
        logFilename = "results/CAM_data_from_person_v2.csv"; // (v2)
    }
    mLogFile.open(logFilename, std::ios::out | std::ios::app);
    if (mLogFile.tellp() == 0) {
        mLogFile <<
"Time_s;Observer_ID;Target_ID;Target_Lat_Raw;Target_Lon_Raw;Target_Spe
ed_Raw" << std::endl;
    }
    mLogTimer = new cMessage("logTimer");
    scheduleAt(simTime() + mLogInterval, mLogTimer);
    mCamReceivedSignal = registerSignal("CamReceived");

```

```

getParentModule()->subscribe(mCamReceivedSignal, this);
// --- Akhir Logika Log Mentah ---

// --- PERUBAHAN UTAMA: Logika Inisialisasi Prediksi (KF) ---
std::string kfLogFilename;
if (nodeType == "Vehicle") {
    kfLogFilename = "results/kf_prediction_log_from_car.csv";
} else {
    kfLogFilename = "results/kf_prediction_log_from_person.csv";
}
mPredictionLogFile.open(kfLogFilename, std::ios::out |
std::ios::app);
if (mPredictionLogFile.tellp() == 0) {
    // Header log baru untuk hasil KF
    mPredictionLogFile <<
"Time_prediction(s);Node_Target;Pred_Lat;Pred_Lon" << std::endl;
}
mPredictionTimer = new cMessage("predictionTimer");
scheduleAt(simTime() + 1.0, mPredictionTimer); // Tetap 1 detik
// --- AKHIR PERUBAHAN ---
}

void TrajektoriAppVerIND::handleMessage(cMessage* msg)
{
    // --- Logika Timer Log Mentah (Sama seperti v1) ---
    if (msg == mLogTimer) {
        logTrajectory();
        scheduleAt(simTime() + mLogInterval, mLogTimer);
        return;
    }

    // --- PERUBAHAN UTAMA: Ganti nama fungsi ---
    if (msg == mPredictionTimer) {
        runPredictions(); // Panggil fungsi prediksi KF
        scheduleAt(simTime() + 1.0, mPredictionTimer);
        return;
    }
    // --- AKHIR PERUBAHAN ---

    delete msg;
}

void TrajektoriAppVerIND::receiveSignal(cComponent* source,
simsignal_t signalID, cObject* obj, cObject* details)
{
    if (signalID != mCamReceivedSignal) {
        return;
    }

```

```

    }
    auto ca_obj = dynamic_cast<CaObject*>(obj);
    if (!ca_obj) {
        return;
    }
    const cPacket* packet = dynamic_cast<const cPacket*>(details);
    simtime_t creationTime = packet ? packet->getCreationTime() :
simTime();
    const auto& cam = *ca_obj->asn1();
    long stationId = cam.header.stationID;
    const auto& basic = cam.cam.camParameters.basicContainer;
    const auto& hfc = cam.cam.camParameters.highFrequencyContainer;
    if (hfc.present !=
HighFrequencyContainer_PR_basicVehicleContainerHighFrequency) {
        return;
    }
    const auto& bvc = hfc.choice.basicVehicleContainerHighFrequency;
    MovementData data;
    data.timestamp = creationTime;
    data.latitude =
static_cast<double>(basic.referencePosition.latitude)/10;
    data.longitude =
static_cast<double>(basic.referencePosition.longitude)/10;
    data.speed_mps = static_cast<double>(bvc.speed.speedValue);

    AgentHistory& history = mOtherNodes[stationId];
    history.history.push_back(data); // Simpan data mentah
    history.lastReceptionTime = simTime();
    history.hasNewData = true;

    // --- PERUBAHAN UTAMA: Logika Update KF ---

    // 1. Jika ini agen baru, buat instance KF
    if (!history.kf_state) {
        history.kf_state = std::make_unique<KalmanFilterIND>();
        // Inisialisasi KF dengan data pertama
        history.kf_state->init(data.latitude, data.longitude,
data.timestamp.dbl());
    } else {
        // 2. Jika agen sudah ada, update KF dengan data baru
        history.kf_state->update(data.latitude, data.longitude,
data.timestamp.dbl());
    }
    // --- AKHIR PERUBAHAN ---

    // Logika pembersihan history lama (Sama seperti v1)

```

```

        while (!history.history.empty() && (simTime() -
history.history.front().timestamp > 5.0)) {
            history.history.pop_front();
        }
    }

// Fungsi logTrajectory sama persis seperti v1
void TrajektoriAppVerIND::logTrajectory()
{
    auto& vdp = getFacilities().get_const<VehicleDataProvider>();
    long myId = vdp.station_id();
    if (!mLogFile.is_open()) return;
    for (auto& [targetId, targetHist] : mOtherNodes) {
        if (targetHist.hasNewData) {
            if (targetHist.history.empty()) {
                continue;
            }
            MovementData latest = targetHist.history.back();
            mLogFile << std::fixed << std::setprecision(12)
                << latest.timestamp.dbl() << ";"
                << myId << ";"
                << targetId << ";"
                << latest.latitude << ";"
                << latest.longitude << ";"
                << latest.speed_mps << std::endl;
            targetHist.hasNewData = false;
        }
    }
    mLogFile.flush();
}

```

```

// --- PERUBAHAN UTAMA: Fungsi Prediksi (menggantikan logCoefficients)
---
void TrajektoriAppVerIND::runPredictions()
{
    simtime_t now = simTime();
    simtime_t prediction_time_target = now + 1.0; // Waktu target
    prediksi

    for (auto const& [targetId, targetHist] : mOtherNodes)
    {
        // Pastikan KF sudah diinisialisasi
        if (targetHist.kf_state && targetHist.kf_state-
>isInitialized())
        {
            // 1. Panggil fungsi predict() dari KF

```

```

        // Kita prediksi 1.0 detik ke depan dari SEKARANG
        auto [pred_lat, pred_lon] = targetHist.kf_state-
>predict(1.0);

        // 2. Log hasil prediksi
        mPredictionLogFile << std::fixed << std::setprecision(12)
            << prediction_time_target.dbl() << ";"
            << targetId << ";"
            << pred_lat << ";"
            << pred_lon << std::endl;
    }
}
mPredictionLogFile.flush();
}
// --- AKHIR PERUBAHAN ---

void TrajektoriAppVerIND::finish()
{
    // --- Logika Finish Log Mentah (Sama seperti v1) ---
    if (mLogFile.is_open()) mLogFile.close();
    cancelAndDelete(mLogTimer);

    // --- PERUBAHAN UTAMA: Cleanup untuk file & timer KF ---
    if (mPredictionLogFile.is_open()) mPredictionLogFile.close();
    cancelAndDelete(mPredictionTimer);
    // --- AKHIR PERUBAHAN ---

    ItsG5BaseService::finish();
}

} // namespace artery

```